

Surface Registration for Computer Integrated Surgery: Closest Points, Rigid ICP and Deformable Registration to a Shape Atlas

Parmida Mazloomi

Programming Assignments 3, 4 and 5 of JHU EN.601.455/655 Computer Integrated Surgery I (Fall 2025)

1 Overview

A bone has been scanned with CT and its surface segmented into a triangle mesh (1568 vertices, 3135 triangles). In the operating room, an optical tracker follows two rigid bodies with LED markers: body B is screwed into the bone, and body A is a pointer whose tip touches points on the bone surface. The goal is the registration F_{reg} from bone-body coordinates to CT coordinates, so that the CT model can be used for navigation. The three assignments build this up in steps [1, 2]:

- **PA3:** compute the pointer tip $\mathbf{d}_k$ in body-B coordinates for each sample, and the closest point $\mathbf{c}_k$ on the mesh with $F_{\text{reg}} = I$; first by brute force, then with a fast spatial search.
- **PA4:** iterative closest point (ICP) registration, alternating matching and rigid registration until F_{reg} converges.
- **PA5:** the bone shape is modelled by a statistical atlas (mean shape plus 6 modes); estimate F_{reg} and the mode weights λ_m together.

The program is written in Python 3.11 with NumPy [11] (arrays, `numpy.linalg.svd` and `lstsq`, which call LAPACK [12]) and Matplotlib [13] for figures. No registration, closest-point or calibration library code is used.

2 Mathematical approach

Frames and the pointer tip. A frame $F = [R, \mathbf{p}]$ maps $\mathbf{x} \mapsto R\mathbf{x} + \mathbf{p}$, with $F_1 F_2 = [R_1 R_2, R_1 \mathbf{p}_2 + \mathbf{p}_1]$ and $F^{-1} = [R^T, -R^T \mathbf{p}]$ [4]. For sample k , the poses $F_{A,k}$ and $F_{B,k}$ are found by registering the body-frame marker positions to the measured ones, and [1]

$$\mathbf{d}_k = F_{B,k}^{-1} F_{A,k} \mathbf{A}_{\text{tip}}, \quad \mathbf{s}_k = F_{\text{reg}} \mathbf{d}_k. \quad (1)$$

Point-set registration. For pairs $(\mathbf{a}_i, \mathbf{b}_i)$ we minimize $\sum_i \|\mathbf{R}\mathbf{a}_i + \mathbf{p} - \mathbf{b}_i\|^2$ with the SVD method of Arun et al. [8, 5]. With centred points $\tilde{\mathbf{a}}_i = \mathbf{a}_i - \bar{\mathbf{a}}$, $\tilde{\mathbf{b}}_i = \mathbf{b}_i - \bar{\mathbf{b}}$, form $H = \sum_i \tilde{\mathbf{a}}_i \tilde{\mathbf{b}}_i^T = U S V^T$ and set

$$R = V \text{diag}(1, 1, \det(VU^T)) U^T, \quad \mathbf{p} = \bar{\mathbf{b}} - R\bar{\mathbf{a}}. \quad (2)$$

Without the diagonal factor, VU^T can be a reflection ($\det = -1$). This happens when the points are coplanar, so that the smallest singular value is zero; flipping the corresponding column of V is then exact, because that column does not affect the fit. With noise it gives the best proper rotation [9]. If two singular values are zero (collinear points), the rotation about the line is undetermined; this cannot occur with the six non-collinear markers used here.

Closest point on a triangle. Following [6], write $\mathbf{a} \approx \mathbf{p} + \lambda(\mathbf{q} - \mathbf{p}) + \mu(\mathbf{r} - \mathbf{p})$ and solve the 2×2 normal equations

$$\begin{bmatrix} \mathbf{e}_1 \cdot \mathbf{e}_1 & \mathbf{e}_1 \cdot \mathbf{e}_2 \\ \mathbf{e}_1 \cdot \mathbf{e}_2 & \mathbf{e}_2 \cdot \mathbf{e}_2 \end{bmatrix} \begin{bmatrix} \lambda \\ \mu \end{bmatrix} = \begin{bmatrix} \mathbf{w} \cdot \mathbf{e}_1 \\ \mathbf{w} \cdot \mathbf{e}_2 \end{bmatrix}, \quad \mathbf{e}_1 = \mathbf{q} - \mathbf{p}, \mathbf{e}_2 = \mathbf{r} - \mathbf{p}, \mathbf{w} = \mathbf{a} - \mathbf{p} \quad (3)$$

by Cramer's rule. If $\lambda, \mu \geq 0$ and $\lambda + \mu \leq 1$, the projection $\mathbf{c} = \mathbf{p} + \lambda\mathbf{e}_1 + \mu\mathbf{e}_2$ is inside the triangle and is the closest point, since $\mathbf{a} - \mathbf{c}$ is normal to the plane. Otherwise the closest point lies on an edge. The lecture's table picks one edge from the sign pattern. When two barycentric coordinates are negative the answer can be on either

edge that meets at the shared vertex, and in a random test a fixed choice was wrong in 13% of those cases (obtuse triangles). We therefore project $\mathbf{a}$ onto all three edges with

$$\text{ProjectOnSegment}(\mathbf{a}, \mathbf{p}, \mathbf{q}) = \mathbf{p} + t^*(\mathbf{q} - \mathbf{p}), \quad t^* = \text{clamp}\left(\frac{(\mathbf{a} - \mathbf{p}) \cdot (\mathbf{q} - \mathbf{p})}{\|\mathbf{q} - \mathbf{p}\|^2}, 0, 1\right) \quad (4)$$

and keep the nearest. This is always correct, and it also handles degenerate triangles, where the 2×2 system is singular. The barycentric weights of the result are returned as well, since PA5 needs them.

Bounding-box tree. Every node of a binary tree holds a group of triangles and the axis-aligned box $[\mathbf{l}, \mathbf{u}]$ around all their corners. The distance from $\mathbf{a}$ to the box, $\|\max(\mathbf{l} - \mathbf{a}, \mathbf{0}, \mathbf{a} - \mathbf{u})\|$, is a lower bound on the distance to any triangle in the node, so if it exceeds the distance β to a surface point already found, the node can be skipped [6]. Since the bound is never larger than the true distance, pruning never removes the true closest point, and the result equals brute force.

Rigid ICP. ICP minimizes $E(F) = \sum_k \text{dist}(F\mathbf{d}_k, \mathcal{M})^2$ by alternating (i) matching, $\mathbf{c}_k = \arg \min_{\mathbf{c} \in \mathcal{M}} \|F\mathbf{d}_k - \mathbf{c}\|$, and (ii) registration of the pairs $(\mathbf{d}_k, \mathbf{c}_k)$ [10, 7]. Each step can only lower E , so ICP descends monotonically to a local minimum.

Statistical shape model. Vertex v of the deformed bone is $\mathbf{m}_v(\lambda) = \mathbf{m}_{0,v} + \sum_{m=1}^M \lambda_m \mathbf{m}_{m,v}$ [2]; the modes are orthonormal over all 4704 coordinates. If $\mathbf{c}_k$ lies on triangle (s, t, u) with barycentric coordinates (ζ_k, ξ_k, ψ_k) , the same barycentric point on the deformed triangle is

$$\mathbf{c}_k = \mathbf{q}_{0,k} + \sum_m \lambda_m \mathbf{q}_{m,k}, \quad \mathbf{q}_{m,k} = \zeta_k \mathbf{m}_{m,s} + \xi_k \mathbf{m}_{m,t} + \psi_k \mathbf{m}_{m,u}, \quad (5)$$

which is linear in λ [3]. With F_{reg} and the matches fixed, the weights solve the linear least-squares problem $\min_{\lambda} \sum_k \|\mathbf{s}_k - \mathbf{q}_{0,k} - \sum_m \lambda_m \mathbf{q}_{m,k}\|^2$.

Linearized combined step. To update pose and shape together, a small correction $\Delta F = [\Delta R(\boldsymbol{\alpha}), \boldsymbol{\epsilon}]$ with $\Delta R\mathbf{s} \approx \mathbf{s} + \boldsymbol{\alpha} \times \mathbf{s}$ is applied to the samples, requiring $\Delta F\mathbf{s}_k \approx \mathbf{c}_k + \sum_m \Delta \lambda_m \mathbf{q}_{m,k}$. Rearranged, this is the handout's equation (7):

$$\mathbf{s}_k \times \boldsymbol{\alpha} - \boldsymbol{\epsilon} + \sum_m \Delta \lambda_m \mathbf{q}_{m,k} \approx \mathbf{s}_k - \mathbf{c}_k, \quad (6)$$

three rows $[\text{skew}(\mathbf{s}_k), -I, \mathbf{q}_{1,k} \cdots \mathbf{q}_{M,k}]$ per sample in the $6 + M$ unknowns. Because $\mathbf{c}_k$ already includes the current weights, the weight unknowns must be the changes $\Delta \lambda = \lambda^{(t+1)} - \lambda^{(t)}$; the handout writes $\lambda^{(t+1)}$, which would count the current deformation twice. The rotation is then made exact with Rodrigues' formula, $\Delta R = I + \sin \theta K + (1 - \cos \theta) K^2$ with $\theta = \|\boldsymbol{\alpha}\|$ and $K = \text{skew}(\boldsymbol{\alpha}/\theta)$, rather than the non-orthogonal $I + \text{skew}(\boldsymbol{\alpha})$.

3 Algorithmic approach

Algorithm 1 Batched bounding-box tree search (`BoundingBoxTree.closest_points`)

Require: query points $\mathbf{a}_1 \dots \mathbf{a}_N$; optional hint triangles (e.g. the previous ICP matches)

- 1: $\beta_k \leftarrow$ distance to the hint triangle, or to the triangles of the leaf reached by always stepping into the nearer child box
 - 2: $P \leftarrow \{(k, \text{root})\}$; $L \leftarrow \emptyset$
 - 3: **while** $P \neq \emptyset$ **do** $\triangleright$ one tree level per pass, vectorized over all pairs
 - 4: keep $(k, n) \in P$ with $\text{dist}(\mathbf{a}_k, \text{box}_n) \leq \beta_k$
 - 5: move pairs with leaf n to L ; replace the others by $(k, \text{left}(n))$ and $(k, \text{right}(n))$
 - 6: **end while**
 - 7: expand L to (query, triangle) pairs; drop those whose triangle box is farther than β_k
 - 8: test the rest with the closest-point-on-triangle routine; keep the nearest per query
-

The tree is built by splitting each node's triangles at the median of their centroids along the longest axis of the centroid spread, until at most 8 triangles remain (1023 nodes, depth 10). For a deforming mesh only the boxes

are refit, children before parents. Keeping β_k fixed during the walk (instead of lowering it as in the lecture’s depth-first search) costs some extra triangle tests but lets each level run as one NumPy operation over all queries.

Algorithm 2 Rigid ICP (`icp.icp`)

Require: tips $\mathbf{d}_k$, search structure, $F_0 = I$; options: warm-up 3, scale 3, floor 0.5 mm, tolerance $\tau = 10^{-5}$ mm

- 1: $\mathbf{s}_k \leftarrow F_0 \mathbf{d}_k$; match $\mathbf{c}_k$; $\eta \leftarrow \infty$
- 2: **for** $n = 1, 2, \dots, 500$ **do**
- 3: **if** $n > 3$: $\eta \leftarrow \max(0.5, \min(\eta, 3 \text{mean}_k \|\mathbf{s}_k - \mathbf{c}_k\|))$ ▷ shrinking threshold
- 4: inliers $\leftarrow \{k : \|\mathbf{s}_k - \mathbf{c}_k\| \leq \eta\}$, widened if needed to keep at least half the pairs
- 5: $F_n \leftarrow \text{REGISTER}(\{\mathbf{d}_k\}, \{\mathbf{c}_k\})$ over the inliers; $\mathbf{s}_k \leftarrow F_n \mathbf{d}_k$; rematch, seeded with the previous triangles
- 6: **stop if** $\max_k \|F_n \mathbf{d}_k - F_{n-1} \mathbf{d}_k\| < \tau$, or the mean residual changed by less than 10^{-9} (relative) for 5 iterations
- 7: **end for**

The motion test bounds the change of both the rotation and the translation of F_{reg} in millimetres at the samples, which is easier to interpret than separate tolerances. The threshold follows the lecture’s suggestion of about $3\varepsilon_n$ after a few iterations [7].

Algorithm 3 Deformable registration (`deformable.deformable_registration`)

- 1: $\lambda \leftarrow 0$; $F_{\text{reg}} \leftarrow \text{ICP to the mean shape}$
- 2: **for** 3 rounds **do** ▷ handout’s first method
- 3: repeat up to 10 times: match on the current mesh; solve the weight least squares; deform; refit the tree
- 4: $F_{\text{reg}} \leftarrow \text{ICP from the current } F_{\text{reg}} \text{ with the shape fixed (tolerance } 10^{-3} \text{ mm)}$
- 5: **end for**
- 6: **repeat** ▷ handout’s second method
- 7: match; solve (7) for $(\alpha, \epsilon, \Delta\lambda)$; $F_{\text{reg}} \leftarrow [\text{Rodrigues}(\alpha), \epsilon] F_{\text{reg}}$; $\lambda \leftarrow \lambda + \Delta\lambda$; refit the tree
- 8: **until** the step moves every sample and vertex by less than 10^{-5} mm

Running the alternation to convergence took about 250 mode steps and 280 rigid ICP iterations per debug set, because rigid motions and modes move the points in similar directions and separate steps zigzag. The combined step treats them together; either route reaches the same weights to within 0.001.

4 Overview of program structure

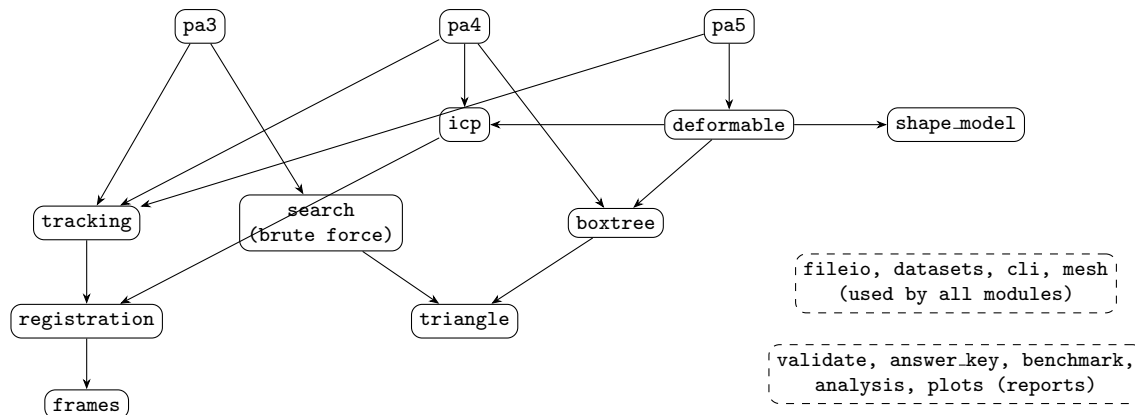


Figure 1: Module hierarchy of the `cisreg` package (arrows: “uses”). The three programs at the top are the command-line entry points; the dashed boxes are shared input and output code and the tools that produce the validation tables and figures.

Module	Main functions: inputs $\rightarrow$ outputs
<code>fileio</code>	<code>read_mesh</code> , <code>read_body</code> , <code>read_samples</code> , <code>read_modes</code> , <code>read_output</code> , <code>read_logfile</code> : file $\rightarrow$ arrays; <code>write_output</code> : points, weights $\rightarrow$ file in the instructor's layout
<code>datasets, cli</code>	file names of each set; <code>load_inputs</code> : (assignment, set) $\rightarrow$ mesh, bodies, readings; shared options
<code>frames</code>	<code>Frame</code> (apply, compose, inverse); <code>rotation_from_vector</code> (Rodrigues), <code>rotation_angle</code> , <code>frame_difference</code>
<code>registration</code>	<code>register_points</code> : paired $(N, 3)$ arrays $\rightarrow$ <code>Frame</code> (Arun SVD with reflection fix)
<code>tracking</code>	<code>body_poses</code> , <code>pointer_tips</code> : bodies, readings $\rightarrow \mathbf{d}_k$
<code>triangle</code>	<code>closest_point_on_triangle</code> : points, corners (broadcast) $\rightarrow$ closest points, barycentric weights
<code>search, boxtree</code>	<code>BruteForceSearch</code> , <code>BoundingBoxTree</code> : <code>closest_points(queries, hint)</code> $\rightarrow$ points, distances, triangles, weights; <code>update_vertices</code>
<code>icp</code>	<code>icp</code> : $\mathbf{d}_k$, search, F_0 , options $\rightarrow F_{\text{reg}}$, $\mathbf{s}_k$, matches, per-iteration history
<code>shape_model</code>	<code>ShapeModel.from_atlas</code> , <code>vertices(λ)</code> , <code>mesh(λ)</code>
<code>deformable</code>	<code>mode_basis</code> , <code>solve_mode_weights</code> , <code>solve_combined_step</code> , <code>deformable_registration</code> $\rightarrow F_{\text{reg}}$, λ , matches, history
<code>pa3, pa4, pa5</code>	command-line programs: <code>python -m cisreg.pa4 --all</code> writes output/, logs and summaries

Each program writes `output/PAV-X-Output.txt` in the instructor's format: a header line, for PA5 a line with the mode weights, then $\mathbf{s}_k$, $\mathbf{c}_k$ and $\|\mathbf{s}_k - \mathbf{c}_k\|$ per sample.

5 Validation approach

Unit tests (508 tests with `pytest`, run on every push by GitHub Actions):

- *Readers*: every file in `data/` is read and its counts checked against its header.
- *Registration*: 50 random rigid transforms with 3 to 40 points are recovered to 10^{-9} ; with noise, the estimate fits at least as well as the true frame and no small perturbation improves it; coplanar point sets that make VU^T a reflection are recovered exactly.
- *Closest point on a triangle*: hand-computed interior, edge and vertex cases above, below and in the plane; collinear and repeated corners; the obtuse two-negative case; and about 800 random point and triangle pairs checked against dense sampling of the triangle.
- *Tree*: distances identical to brute force for the pointer tips of all 33 data sets, random near and far points, deliberately wrong hints and a deformed mesh; different triangles are accepted only for exact ties at shared edges or vertices.
- *ICP and PA5*: synthetic surface points with known F_{reg} (and weights), with and without noise and injected outliers; the RMS residual never increases without rejection; one combined step recovers a known small correction; mode weights are recovered exactly from exact matches.

Debug sets. Our output files are compared with the instructor's Output and Answer files using $e_{s,k} = \|\mathbf{s}_k - \mathbf{s}'_k\|$ and $e_{c,k} = \|\mathbf{c}_k - \mathbf{c}'_k\|$ (max and RMS = $\sqrt{\text{mean}_k e_k^2}$), and $\max_m |\lambda_m - \lambda'_m|$ for the weights. Since $\mathbf{s}_k = F_{\text{reg}} \mathbf{d}_k$, the instructor's F_{reg} can be recovered by registering our $\mathbf{d}_k$ onto the reference $\mathbf{s}'_k$; both frames are then scored by the sum of squared distances to the surface. Full tables are in `results/pa3_validation.md`, `pa4_validation.md` and `pa5_validation.md`.

Rounding floor. All files are rounded to 0.01 mm. Two independent roundings give a 3D RMS difference of $\sqrt{3 \cdot 2 \cdot 0.01^2 / 12} = 0.0071$ mm, and a Monte Carlo test (readings perturbed by ± 0.005 mm) shows that the input rounding moves $\mathbf{d}_k$ by 0.0073 mm RMS, about 0.010 mm in total. This is the level at which two correct programs can agree.

6 Results

Debug sets. Table 1 summarizes the agreement. On the noise-free sets everything agrees at the rounding floor. On the noisy PA4 sets our $\mathbf{s}_k$ differ from the instructor's by up to 0.05 mm, but our F_{reg} has a lower sum of squared

distances (1.578 against 1.602 mm² for E, 1.477 against 1.498 for F), so the difference is a different stopping point of the instructor’s program, not an error. The noise-free PA5 weights agree to 0.01 to 0.05; perturbing the readings at their rounding level moves them by a standard deviation of 0.014 to 0.041, so the rounded data determine the weights only to that level.

Table 1: Agreement with the instructor’s Output files on the debug and demo sets (mm and degrees).

	max e_s	RMS e_s	max e_c	F_{reg} difference	max weight diff.
PA3, A to F	0.010 to 0.022	0.008 to 0.011	0.014 to 0.020	$(F_{\text{reg}} = I)$	
PA4, A to D and demos (noise-free)	0.014 to 0.022	0.007 to 0.009	0.014 to 0.017	$\leq 0.004^\circ, \leq 0.002$	
PA4, E and F (0.1 mm noise)	0.051	0.021 to 0.024	0.045 to 0.051	$0.02 \text{ to } 0.03^\circ, \leq 0.014$	
PA5, A to D (noise-free)	0.014 to 0.025	0.008 to 0.010	0.014 to 0.025	$\leq 0.0044^\circ, \leq 0.0012$	0.011 to 0.046
PA5, E and F (0.1 mm noise)	0.028 to 0.033	0.012 to 0.014	0.025	$\leq 0.021^\circ, \leq 0.004$	0.20 to 0.34

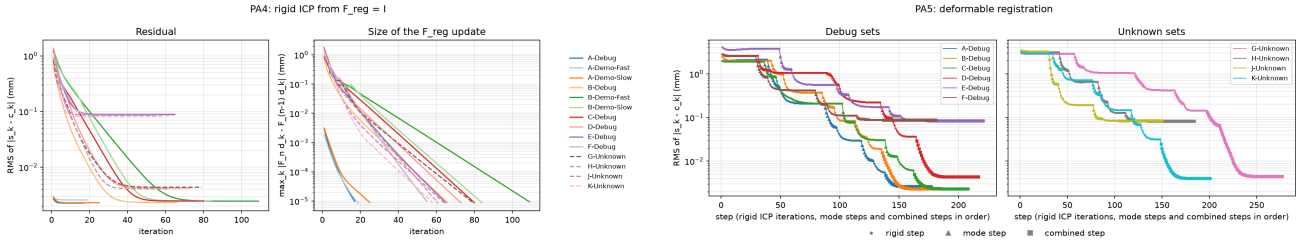


Figure 2: Left: PA4 RMS residual and size of the F_{reg} update per ICP iteration; convergence is linear, to a floor of 0.0025 mm (noise-free) or 0.085 mm (noisy). Right: PA5 residual per step. Rigid phases plateau, mode steps (triangles) cause sharp drops, and combined steps (squares) finish.

Unknown sets. Tables 2 and 3 give the results. The outputs were committed before the instructor’s log files were opened. A later comparison (`results/answer_key_comparison.md`) found every unknown set registered correctly: within 0.006° and 0.003 mm of the true F_{reg} on the noise-free sets and within 0.075° and 0.018 mm on the noisy ones, with errors comparable to the instructor’s program. The noise-free PA5 weights are within 0.1 of the true values; with the rounded readings our solution fits the data better than the true parameters do (for example 0.00297 against 0.00360 mm² for PA5-G), so the remaining difference is due to the input rounding. The log’s “RMS residual” turned out to be the mean of $\|\mathbf{s}_k - \mathbf{c}_k\|$, which is why both are reported.

Table 2: PA3 and PA4 unknown sets. Residuals $\|\mathbf{s}_k - \mathbf{c}_k\|$ in mm; F_{reg} as rotation angle, rotation vector (rad) and translation (mm).

Set	N	Iter.	Mean	RMS	Max	Angle	Rotation vector; translation
PA3-G	20		1.663	1.924	3.944		$F_{\text{reg}} = I$
PA3-H	20		1.538	1.700	3.010		$F_{\text{reg}} = I$
PA3-J	20		2.318	2.709	5.405		$F_{\text{reg}} = I$
PA4-G	200	79	0.0033	0.0044	0.014	2.395°	$(0.02497, 0.02999, 0.01496); (1.001, -1.500, 0.251)$
PA4-H	200	61	0.0033	0.0041	0.012	0.005°	$(0.00005, 0.00002, 0.00005); (1.001, 3.001, -2.000)$
PA4-J	200	64	0.0667	0.0863	0.295	0.057°	$(-0.00026, -0.00040, -0.00086); (3.006, -1.498, 0.505)$
PA4-K	200	58	0.0632	0.0810	0.236	0.830°	$(0.00881, 0.00478, -0.01045); (-1.489, 2.510, 0.002)$

Table 3: PA5 unknown sets: mode weights, residuals (mm) and F_{reg} .

Set	N	λ_1	λ_2	λ_3	λ_4	λ_5	λ_6	Mean / RMS	F_{reg} angle; translation
G	150	-96.381	160.135	-81.887	-78.811	126.233	141.516	0.0036 / 0.0044	$1.342^\circ; (-0.999, 1.001, -1.499)$
H	400	84.172	95.279	-81.356	-134.684	37.325	110.282	0.0633 / 0.0813	$0.036^\circ; (1.008, -0.995, 2.003)$
J	400	-5.233	-144.473	-57.066	-132.199	15.544	-108.204	0.0647 / 0.0838	$1.111^\circ; (1.495, 1.012, 1.989)$
K	150	54.190	153.113	-60.647	9.827	114.480	-43.526	0.0030 / 0.0040	$0.003^\circ; (0.002, 2.000, -0.999)$

Error analysis. Figure 3 and `results/error_analysis.md` summarize four experiments. (i) ICP started from misaligned poses on PA4-B reaches the right minimum in every trial up to 60° and 30 mm, in 92 to 100% at 75° , and rarely beyond 120° ; the course data start within 3.5° and 6 mm. (ii) Marker noise propagates linearly: at $\sigma = 0.1$ mm, F_{reg} is good to about 0.06° and 0.03 mm, but the PA5 weights only to about 1 unit with 150 samples, so the weights are the least certain output. (iii) The noise-free PA5 sets need all 6 modes; leaving out any mode leaves residuals of 0.3 to 0.9 mm. (iv) The tree is 2.8 times faster than brute force for one point and 20 to 100 times faster for the 75 to 400 points of a data set, with identical results.

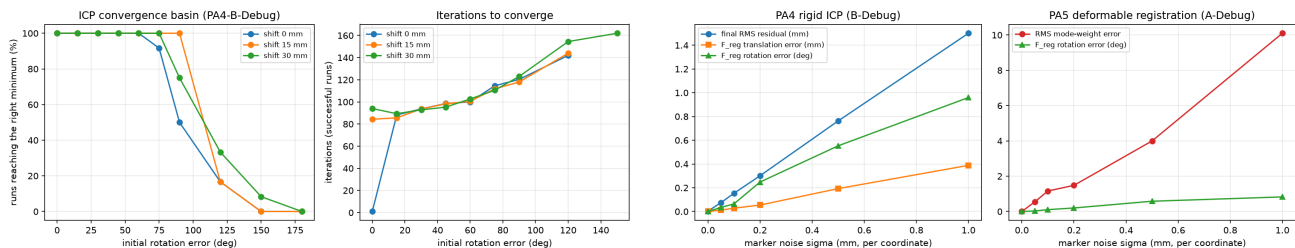


Figure 3: Left: share of ICP runs reaching the right minimum against the initial rotation error (and iterations needed). Right: errors against added marker noise for PA4 (rigid) and PA5 (deformable).

Discussion. All three programs reproduce the instructor’s debug results at the level permitted by the rounding of the data files, and their fits are at least as good as the instructor’s in the least-squares sense. The main limitations are those of ICP itself: it needs an initial pose within roughly 75° , it converges linearly (tens to about a hundred iterations here), and it weights all pairs equally. The combined PA5 step converges faster than the alternation because it handles the coupling between pose and shape. Possible extensions are point-to-plane matching for faster convergence, a weighted or robust objective, and a prior on the mode weights (a penalty $\sum_m \lambda_m^2 / \sigma_m^2$) to stabilize the weights when the data are noisy.

Statement of work. This is an individual portfolio project by Parmida Mazloomi, carried out after the course and not submitted for credit. The problem, data and reference outputs are by R. H. Taylor and the CIS I teaching staff; no code from other students’ solutions was used.

References

- [1] R. H. Taylor, “Programming Assignments 3 and 4,” EN.601.455/655 Computer Integrated Surgery I, Johns Hopkins University, Fall 2025.
- [2] R. H. Taylor, “Programming Assignment 5,” EN.601.455/655, Fall 2025.
- [3] R. H. Taylor, “Notes for PA5: Deformable registration to a statistical shape model,” lecture notes, 2025.
- [4] R. H. Taylor, “Frames and frame transformations,” CIS I lecture notes, 2025.
- [5] R. H. Taylor, “Point cloud to point cloud rigid transformations,” CIS I lecture notes.
- [6] R. H. Taylor, “Finding point-pairs,” CIS I lecture notes, 2010 to 2022.
- [7] R. H. Taylor, “Registration, part 1” (iterative closest point and outline of a practical ICP code), CIS I lecture notes, 2025.
- [8] K. S. Arun, T. S. Huang and S. D. Blostein, “Least-squares fitting of two 3-D point sets,” *IEEE Trans. PAMI*, 9(5):698-700, 1987.
- [9] S. Umeyama, “Least-squares estimation of transformation parameters between two point patterns,” *IEEE Trans. PAMI*, 13(4):376-380, 1991.
- [10] P. J. Besl and N. D. McKay, “A method for registration of 3-D shapes,” *IEEE Trans. PAMI*, 14(2):239-256, 1992.
- [11] C. R. Harris et al., “Array programming with NumPy,” *Nature*, 585:357-362, 2020.

- [12] E. Anderson et al., *LAPACK Users' Guide*, 3rd ed., SIAM, 1999.
- [13] J. D. Hunter, "Matplotlib: a 2D graphics environment," *Computing in Science & Engineering*, 9(3):90-95, 2007.